IncidentFlow DigitalOcean Proof

A short-lived cloud deployment proving Terraform, Ansible, Docker Compose, and a working incident workflow.



Terraform + Ansible + Docker Compose

Cloud proof cost: about $0.06

IncidentFlow / Terraform / Ansible / DigitalOcean

Short-lived cloud environment

The goal was proof, not permanent hosting: create the smallest useful VM, test the workflow, capture evidence, then shut it down.

01 PROVISION
Terraform creates cloud resources
Droplet, SSH key, firewall, and output URLs

02 CONFIGURE
Ansible prepares the server
Docker, environment file, Compose stack, and image pulls

03 RUN
Five containers start cleanly
API, PostgreSQL, user UI, support UI, and admin UI

04 VERIFY
Readiness and workflow checks pass
user report -> support resolve -> admin metrics

05 DESTROY
Resources are removed after evidence
cost stays low because the proof is short-lived

Terraform created the infrastructure

The cloud state exposed the public IP, site URLs, Droplet ID, and managed resources needed for the proof.

```
PS D:\Taskroom-project-RB\infrastructure\terraform\environments\digitalocean-proof> terraform output
admin_site_url = "http://164.92.128.197:8086"
ansible_inventory_line = "incidentflow ansible_host=164.92.128.197 ansible_user=root ansible_ssh_private_key_file=~/.ssh/
incidentflow_do_rsa"
droplet_id = "605981791"
public_ipv4 = "164.92.128.197"
support_site_url = "http://164.92.128.197:8085"
user_site_url = "http://164.92.128.197:8084"
PS D:\Taskroom-project-RB\infrastructure\terraform\environments\digitalocean-proof> terraform state list
digitalocean_droplet.incidentflow
digitalocean_firewall.incidentflow
digitalocean_ssh_key.incidentflow
PS D:\Taskroom-project-RB\infrastructure\terraform\environments\digitalocean-proof>
```

Output confirmed: user :8084, support :8085, admin :8086

State list showed DigitalOcean Droplet, firewall, and SSH key resources.

Ansible deployed the stack

The playbook prepared the server, wrote the application environment, pulled images, started Compose, and verified all three sites.

```
pc@DESKTOP-V42UHM6:/mnt/d/Taskroom-project-RB/provision/ansible$ ansible-playbook   -i inventory/digitalocean-proof.ini   -e @group_vars/incidentflow.yml   playbooks/deploy-incidentflow.yml
[WARNING]: Ansible is being run in a world writable directory (/mnt/d/Taskroom-project-RB/provision/ansible), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/
ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
[WARNING]: Found both group and host with same name: incidentflow

PLAY [Deploy IncidentFlow on a Docker host] ************************************************************************

TASK [Gathering Facts] *********************************************************************************************
[WARNING]: Host 'incidentflow' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be di
scovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [incidentflow]

TASK [docker_host : Install Docker packages from Ubuntu repositories] **********************************************
ok: [incidentflow]

TASK [docker_host : Enable and start Docker] ***********************************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Create IncidentFlow application directory] ********************************************
ok: [incidentflow]

TASK [incidentflow_compose : Write IncidentFlow environment file] **************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Write cloud Docker Compose file] ******************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Pull IncidentFlow images] *************************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Start IncidentFlow stack] *************************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Verify user site readiness] ***********************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Verify support site readiness] ********************************************************
ok: [incidentflow]

TASK [incidentflow_compose : Verify admin site readiness] **********************************************************
ok: [incidentflow]

PLAY RECAP *********************************************************************************************************
incidentflow               : ok=11   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Idempotence result: ok=11, changed=0, failed=0

That second clean run is the proof that the automation can be repeated safely.

Containers healthy on the Droplet

The running server showed every service up and healthy, with public ports mapped only for the three portals.

```
pc$DESKTOP-V42UHN6:/mnt/d/Taskroom-project-RB/provision/ansible$ cd /mnt/d/Taskroom-project-RB/provision/ansible

export ANSIBLE_HOST_KEY_CHECKING=False
export ANSIBLE_ROLES_PATH=./roles

ansible \
  -i inventory/digitalocean-proof.ini \
  incidentflow \
  -m command \
  -a "docker compose -f /opt/incidentflow/compose.yaml ps"
[WARNING]: Ansible is being run in a world writable directory (/mnt/d/Taskroom-project-RB/provision/ansible), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/
ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
[WARNING]: Found both group and host with same name: incidentflow
[WARNING]: Host 'incidentflow' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be di
scovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
incidentflow | CHANGED | rc=0 >>
NAME                           IMAGE                                      COMMAND                  SERVICE           CREATED          STATUS                    PORTS
incidentflow-admin-frontend-1  momo1029/incidentflow-admin-ui:latest      "/docker-entrypoint.…"   admin-frontend    About an hour ago  Up About an hour (healthy)  0.0.0.0:8086->80/tcp
incidentflow-backend-1         momo1029/task-manager-api:latest           "docker-entrypoint.s…"   backend           About an hour ago  Up About an hour (healthy)  3001/tcp
incidentflow-database-1        postgres:17-alpine                         "docker-entrypoint.s…"   database          About an hour ago  Up About an hour (healthy)  5432/tcp
incidentflow-support-frontend-1  momo1029/incidentflow-support-ui:latest  "/docker-entrypoint.…"   support-frontend  About an hour ago  Up About an hour (healthy)  0.0.0.0:8085->80/tcp
incidentflow-user-frontend-1   momo1029/incidentflow-user-ui:latest       "/docker-entrypoint.…"   user-frontend     About an hour ago  Up About an hour (healthy)  0.0.0.0:8084->80/tcp
```

BACKEND

3001

internal API

USER

8084

public portal

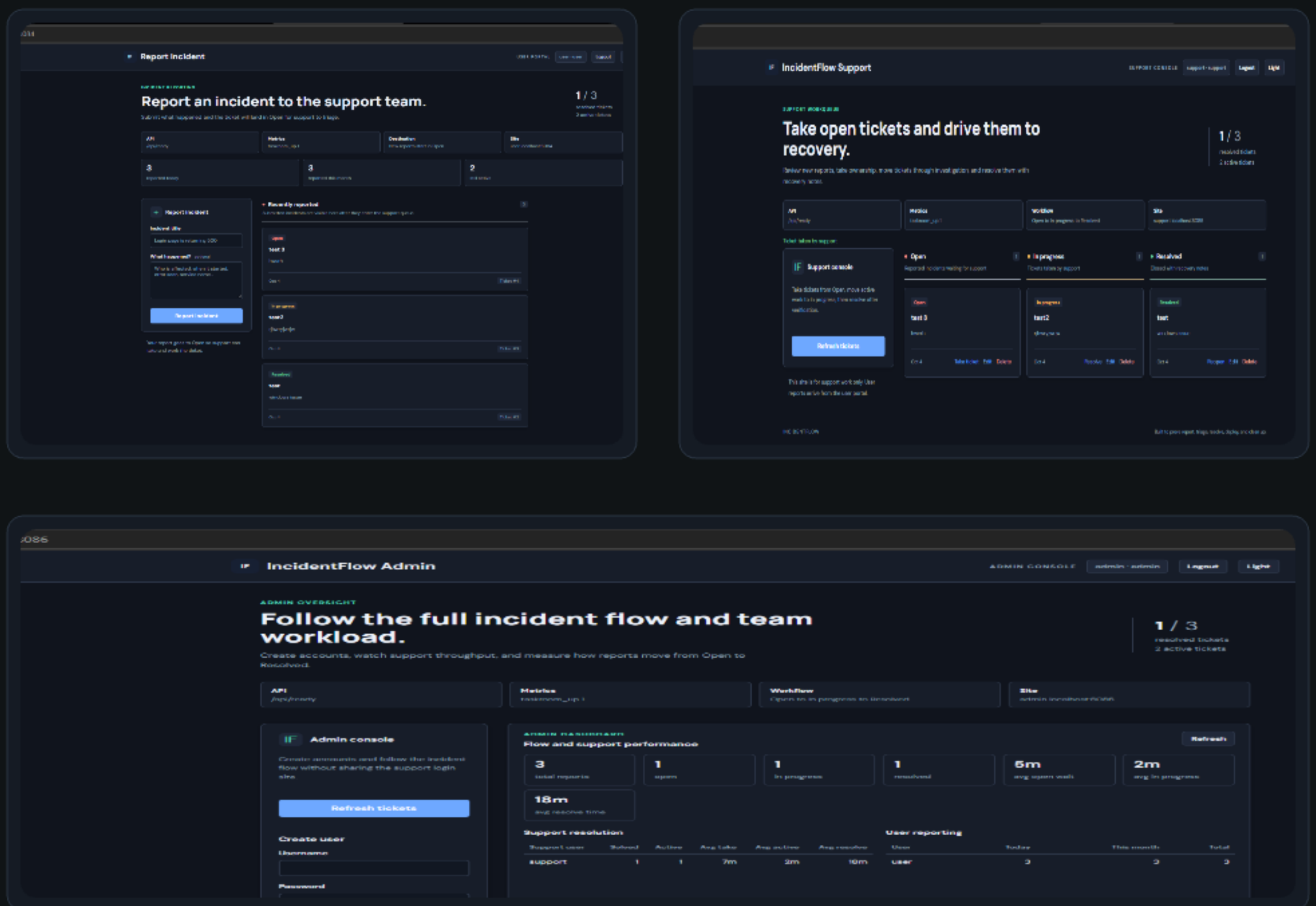
SUPPORT + ADMIN

8085 / 8086

separate sites

Three portals, one cloud VM

IncidentFlow runs as separate web entry points so users, support, and admins do not share the same console.



User reports incidents. Support takes and resolves tickets. Admin follows the whole flow.

IncidentFlow / Terraform / Ansible / DigitalOcean

Admin dashboard measured the flow

The admin view tracked active tickets, resolved work, support ownership, and split timing for Open and In progress stages.

TOTAL REPORTS

3

submitted in proof

OPEN WAIT

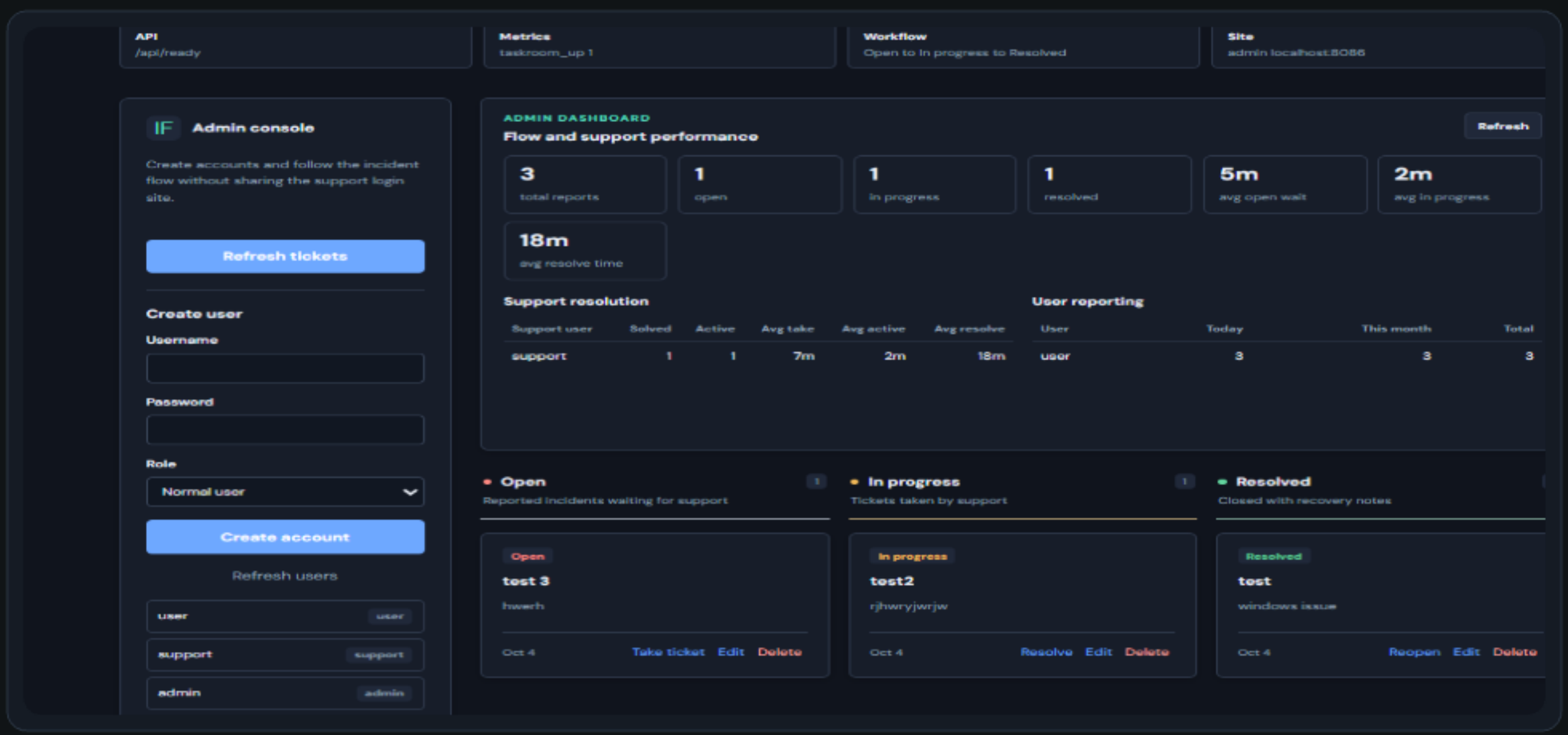
5m

avg open stage

IN PROGRESS

2m

avg active stage



This proves the app is more than static hosting: it has roles, data, workflow, and metrics.

IncidentFlow / Terraform / Ansible / DigitalOcean

Proof first, then AWS

DigitalOcean was the cheapest practical checkpoint: prove the deployment discipline, then repeat it on AWS with the same cost-control rule.

DONE **Terraform infrastructure created and recorded**
public IP, site URLs, Droplet ID, state list

DONE **Ansible deployment repeated cleanly**
second run: ok=11, changed=0, failed=0

DONE **Application workflow tested live**
user report -> support resolution -> admin metrics

NEXT **AWS proof with small resources only**
run, screenshot, document, destroy

Rule: small first, prove the skill, capture evidence, destroy resources.